

Programación Funcional

Paradigmas de la Programación

Grado en Ingeniería Informática

Segundo Semestre

ACTIVIDAD DE EVALUACIÓN CONTINUA AEC1

Prof: Miguel Angel Chicchón
Madrid, Curso 2026-2027

udima
UNIVERSIDAD A DISTANCIA DE MADRID
"La Universidad cercana"

1. Objetivos de aprendizaje

La realización de esta actividad de aprendizaje va a permitir al estudiante el conocimiento e interacción con un lenguaje del Paradigma Funcional

El objetivo que se pretende conseguir es que el estudiante entienda el Paradigma Funcional, a través de cualquiera de los lenguajes de dicho paradigma: Hope, LISP, Haskell... Es posible usar otros lenguajes (Python, c#, Java, Scala, Swift, Kotlin, f#, Rust,…) siempre **y cuando la aproximación sea funcional**. Opcionalmente, se podrá realizar en más de un lenguaje.

La evaluación de este trabajo tendrá en cuenta los siguientes puntos:

- Correcta explicación de las soluciones planteadas.
- Correcta resolución de los ejercicios planteados.
- Correcta entrega: nombre del fichero, entrega no retrasada, formato del fichero correcto.
- Correcta implementación: utilización de variables correctas y con el mismo nombre que se indica en el enunciado.

2. Descripción de la actividad

La actividad consiste en resolver los ejercicios que se plantean a continuación. Se deben implementar en un lenguaje que pertenezca al paradigma funcional. Crear las funciones auxiliares que sean necesarias.

1. Función **imprimir_bienvenida** que escribe "¡Bienvenidos al Paradigma Funcional: Inmutabilidad y Funciones Puras!" en la pantalla.

2. Función **longitud_lista** que, dada una lista de cualquier tipo, devuelve su número de elementos. Restricción: Se debe implementar obligatoriamente mediante recursividad pura y sin utilizar funciones integradas del lenguaje (como length, len o count).

   Ejemplo: longitud_lista([4, 8, 15, 16, 23, 42]) = 6

3. Función e**ncontrar_minimo** que, dada una lista de números devuelva el valor mínimo.

   Ejemplo: encontrar_minimo([9, 3, 7]) = 3

4. Función e**ncontrar_minimo_y_posicion** que, dada una lista de números, devuelva el valor mínimo junto a la posición (contando desde 1) en la que se encuentra dicho valor (es decir, devuelva una tupla)

Ejemplo: encontrar_minimo_y_posicion([9, 1, 7, 3]) = (1, 2)

(puesto que 1 es el mínimo y se encuentra en la segunda posición).

5. Trata de reutilizar alguna de las funciones anteriores, **refactorizandola** para generalizarla si es preciso, para crear la función **sumar_max_y_min** que a partir de una lista de enteros, devuelva la suma del mayor de los elementos y el menor de ellos. E.g. sumar_max_y_min ([2,6,10]) = 10 + 2 = 12

6. Función **obtener_caracter_n** que tome una lista de caracteres y un entero, y devuelva el carácter en la posición indicada por el entero. E.g. obtener_caracter_n(['a','b','c','d'], 2)= 'b'

   (nótese que la posición inicial es la 1)

7. Función **obtener_trinomio_n** que tome una lista de trinomios y un entero, y devuelva el trinomio en la posición indicada por el entero. E.g. obtener_caracter_n([(1,2,3),(4,5,6)], 2)= (4,5,6)

   (nótese que la posición inicial es la 1)
   ¿Sería posible generalizar esta función para cualquier otro tipo de dato, además de trinomios?

8. Función **duplicar_lista** que tome una lista de enteros y devuelva una lista con el doble de cada uno de los valores originales. Ejemplo: duplicar_lista([1,2,3,4]) = [2,4,6,8]

   Se requiere implementar dos versiones:
   a) Utilizando recursividad.
   b) Empleando la función map (o similar) junto con una función lambda.
   c) ¿Qué es una función lambda y qué ventajas ofrece en el contexto de la programación funcional?

9. Función **repetir_accion_n.** Toma una función como parámetro y un entero n, y ejecuta dicha acción n veces de forma estrictamente recursiva (sin bucles for o while). Demuestra su uso invocando imprimir_bienvenida del Ejercicio 1 tantas veces como indique el usuario.

10. Herramientas de big data como *Hadoop* o *Spark* usan la combinación de funciones **map** y **reduce** para procesar datos de manera distribuida, es decir, en múltiples nodos.

a) ¿Por qué crees que usan estas dos funciones, en vez de iterar por todos los elementos (e.g. con un bucle for)? ¿Qué ventajas consideras que puede tener esta manera de abordar la solución usando el paradigma funcional?

b) Investiga y describe las características del lenguaje escogido. ¿Cómo es el tipado que utiliza?¿Qué crees que implica?¿Por qué crees que todos los lenguajes modernos han incorporado la programación funcional?

11. Instrucciones para la realización y entrega de la actividad

Son las siguientes:

- La actividad cumplimentada se envía al profesor a través del Buzón de entrega de esta actividad. En ese mismo buzón aparece la fecha límite de entrega. Los retrasos en la entrega serán penalizados según se indica en la Guía Docente.

- Las soluciones planteadas deben estar bien codificadas y claras.

- El documento a entregar consiste en un fichero *.lsp, *.hop, *.hs, etc. según en lenguaje en el que esté escrito, en el que el estudiante entregará el código de cada ejercicio y los comentarios asociados a éste. Este fichero deberá ser nombrado como con el nombre y los apellidos del estudiante separados por guión bajo y seguidos de AEC1, es decir, siguiendo la siguiente notación:

<nombre>_<apellido1>_<apellido2>_AEC<numeroActividad>.<extensión>

Por ejemplo, si el estudiante es Sinesio Sanchez Pérez y el código está en LISP, deberá enviar un fichero nombrado como sinesio_sanchez_perez_AEC1.lsp.

- El archivo debe contener una cabecera, usando los comentarios del lenguaje, que indique:

  ; **Paradigmas de la Programación**
  ; AEC1
  ; Nombre: <nombre><apellido1><apellido2>

  así mismo, cada ejercicio debe de estar debidamente identificado mediante otro comentario. E.g.

  ; Ejercicio 2

- Adicionalmente, la contestación a las preguntas se debe indicar en cada ejercicio, mediante comentarios.

- La calificación obtenida, previa corrección y calificación por parte del profesor, se podrá consultar en el apartado CALIFICACIONES del Aula Virtual dos semanas después a partir de la terminación del plazo de entrega de esta actividad.